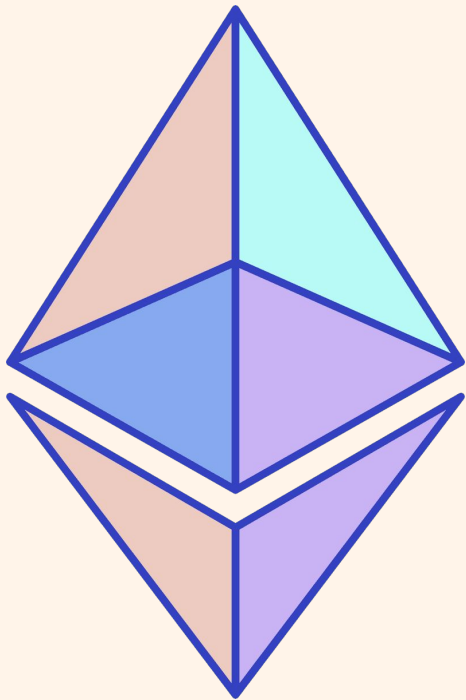




L1-zkEVM breakout: Formal Verification

Alexander Hicks
Ethereum Foundation Research

February 11th, 2026

RISC-V zkVM Track

Verify the arithmetization of RISC-V zkVMs. Ensure polynomial constraints exactly encode RISC-V instruction semantics based on the official Sail RISC-V specification.

EVM Track

Show that an EVM running on RISC-V correctly implements the EVM specification. Focus on performant implementations compilable to RISC-V zkVMs.

Cryptography Track

Verify the specification, proofs of security, and implementation of proof systems and cryptographic primitives used by zkVMs.

verified-zkevm.org



TL;DR zkVM Track

- **Official RISC-V specification available in Lean**
 - **New: General purpose Sail -> Lean extraction**
 - **Extraction also exists in various states for Rocq, HOL4, Isabelle**
- **RISC-V circuit verification ongoing, aiming for Glamsterdam**
 - **SP1**  (caveat: memory consistency is pending)
 - **OpenVM** 
 - **Others coming asap**
 -  **some assumptions regarding lookups**
- **Precompiles**
 - **We are able to verify precompiles e.g. plonky3 keccak done in Rocq**
 - **Issue with precompiles is rather the overall number across all zkVMs**
 - **Verifying autoprecompiles is also ongoing**
- **Misc: Lean DSLs, LLZK, ...**



- **2025: Verification of opcodes on RISC-V (revm -> riscv32im-succinct-zkvm-elf) against KEVM**
- **Current aim: need to update to standardized 64bit RISC-V target, extend to the guest program/STF as a whole, support all relevant guest programs**
 - Looking at options
 - Balance costs of doing all this vs assumption of client diversity
 - Canonical guest program with verified compilation?

- **ArkLib: formally verified executable specifications for proof systems**
- **Hax: Rust -> Lean extraction to verify Rust verifiers**
 - **Challenge: complexity of (features used in) Rust libraries**
- **Misc: try to compile Lean via optimised MLIR dialects to see if we can get faster code than Rust and skip verifying Rust in this instance**



Takeaway

- Progress on each track is ongoing
 - Mix of short term targets (e.g. RISC-V circuit verification) with longer term targets (e.g. formal specs for proof systems, greater integration of formal verification in development process with Lean DSLs, SMT tools, ...)
 - Aiming to be robust to changes in the stack
- Expect certain artifacts (RISC-V circuits) to be verified by Glamsterdam, but not treating this as a blocker either way
 - Using optional proofs period to establish robust maintenance practices
- verified-zkevm.org was recently updated, greater documentation coming soon
- Telegram: <https://t.me/+aoXe0tBozqEwODk0>